

Code review — online-school-vanilla-java

Runde de review pe proiectul tău, cea mai nouă sus.

Documentul se actualizează la fiecare rundă. Runda curentă sus, istoricul jos.

Runda 4 — commit 1bfb58b

Data: 2026-09-09 **Ce acoperă:** a29bce4 („View();”) + 518f740 + 1bfb58b — meniul interactiv, Enrollment + EnrollmentRepository , CardStudent mutat în pachetul lui.

Compilează curat (javac pe toate sursele, 0 erori). Toate constatările de mai jos sunt **reproduse prin rulare**, pe o copie — fișierele tale nu au fost atinse.

Bilanț față de runda 3

#	Ce era	Status
B1	Main crapă la orice rulare	✓ închis — Main nu mai adaugă orbește
B3	CourseRepository() cu constructor gol	✓ închis — loadData() + CRUD
B4	addBook suprascria id-ul primit	✓ închis, elegant
B5	carte la doi studenți deodată	✓ închis prin eliminare — FK pe carte
M1	relațiile nu se persistau	✓ închis — a 4-a coloană în books.txt
C1	câmpuri de test în Student	✓ închis
B2	unicitate care se poate ocoli la update()	● NEATINS, a doua oară
M5, C3, C4	Book în pachet greșit, excepții I/O, snake_case	● neatins

Decizia de la B5 e cea corectă și vreau să fie explicit spus: ai renunțat la `Student.arrBooks` + `Book.student` și ai pus `studentId` pe carte. Asta e exact cum funcționează o bază de date relațională — capătul „many” ține cheia. Ai scăpat dintr-o lovitură de sincronizarea bidirecțională și de persistarea legăturii.

Și ceva mai important: `EnrollmentRepository` e cel mai curat cod pe care l-ai scris până acum. `addEnrollment()` (:34–36) verifică perechea `(studentId, courseId)` **înainte** de `add`, `deleteById` aruncă când nu găsește, `indexOf` returnează `-1`, `save()` rescrie tot. Reține asta, pentru că paza aia din `addEnrollment` e **exact** verificarea care lipsește din B2. Ai scris-o corect când ai avut mâna liberă.

● Critice

B9 — meniul 7 face exact invers decât scrie: te DEZ-înscirie

`src/app/View.java:63–65` · `src/app/View.java:88` · `src/app/View.java:115`

Reprodus. Sunt Alexandra (`m6n–52–67`), aleg 5 (înscrierile mele), apoi 7 („inscrie-te la un curs”), cer „Java Programming”, apoi din nou 5:

```
7-inscrie-te la un curs:      <- meniul
Java Programming             <- inainte: 2 cursuri
Web Development
...
Introdu numele cursului:     <- ce zice de fapt case 7
Java Programming
...
Web Development              <- dupa: 1 curs. M-a SCOS de la Java Prog.
```

case 7: cheamă `removeCourse()`. `registerForCourse()` (:115–129) nu e chemată de nicăieri — am căutat în tot proiectul, zero apeluri. Deci **nu există nicio cale prin care aplicația să înscrie pe cineva la un curs**, iar singurul buton care promite asta face opusul, în tăcere: `removeCourse` nu tipărește nimic la succes.

DE CE: ai scris `registerForCourse()` prima, apoi ai scris `removeCourse()`, și când ai legat-o în `switch` ai pus-o pe cea la care lucrei în ultimul `case` liber. Compilatorul nu are cum să te avertizeze: o metodă `public` nechemată e cod perfect valid, nu e o eroare — Java nu știe că ai *intenționat* s-o chemi. Singura plasă care prinde asta e să rulezi meniul de la 1 la 7 după fiecare metodă nouă și să

verifici că ce scrie pe buton e ce se întâmplă. IntelliJ ți-o și arată: numele metodei nefolosite apare gri deschis în editor.

FIX: `case 7: registerForCourse();` și un `case 8: removeCourse();` cu textul lui în `menu()`. Plus un `System.out.println(...)` la finalul lui `removeCourse` — o operație de scriere care nu spune nimic e o operație pe care utilizatorul o repetă.

B10 — redenumești o carte, iar la repornire numele vechi e înapoi

`src/app/book/repository/BookRepository.java:104-126` (lipsește `save()`)

Reprodus. Redenumesc „House of the Dragon” → „House of the Dragon 2”, apoi construiesc un `BookRepository` nou (= ce se întâmplă la următoarea pornire a aplicației):

```
in memorie dupa update:
    The Pragmatic Programmer
    A Song of Ice and Fire
    House of the Dragon 2      <- s-a schimbat
dupa restart (repo nou, citit din books.txt):
    The Pragmatic Programmer
    A Song of Ice and Fire
    House of the Dragon       <- numele vechi. Modificarea s-a pierdut
```

Asta e o **regresie pe care ai introdus-o tu** în `a29bce4`. Versiunea de dinainte, `updateBook(Book)`, avea ultimele două linii `books.set(index, book); save();`. Când ai rescris metoda ca `updateBookName(oldName, newName, studentId)`, ai păstrat validările și ai pierdut `save()`.

DE CE modificarea se vede totuși în memorie: `findBooksByStudentId()` returnează o **listă nouă**, dar cu **aceleași obiecte `Book`** înăuntru — copiază referințele, nu obiectele. Deci `book.setBookName(newName)` chiar modifică obiectul din `this.books`. De-asta pare că funcționează cât timp aplicația e pornită. Doar că `this.books` e o listă din RAM; fișierul de pe disc nu se schimbă singur când se schimbă un obiect din RAM. `save()` e singurul loc din toată clasa care scrie pe disc — dacă nu-l chemi, tot ce ai făcut trăiește până la primul `exit`.

Și mai e o capcană în aceeași metodă: linia 107, `List<Book> books = findBooksByStudentId(studentId);` — variabila locală `books` **ascunde câmpul `this.books`**. În restul metodei, `books` nu mai înseamnă „toate cărțile”, ci „copia cu cărțile studentului”. Aici n-a stricat

nimic pentru că obiectele sunt aceleași, dar dacă mâine scrii `books.remove(...)` în metoda asta, ștergi din copie și fișierul rămâne neatins — același bug, doar că mai greu de văzut.

FIX: `save();` înainte de `return book;`. Și redenumeste variabila locală în `studentBooks`, ca `books` să însemne un singur lucru în toată clasa.

B2 — RESTANȚĂ, a doua oară: unicitatea la `update()` se poate ocoli

`src/app/student/repository/StudentRepository.java:69-74` .

`src/app/course/repository/CourseRepository.java:77-85` (aici lipsește complet)

Reprodus, ambele direcții, ca să se vadă exact când cedează:

```
directia 1: modific pe ANDREI (poz 1) sa aiba emailul lui BOGDAN (poz 0)
    blocat: Email already used: bogdan.popescu@example.com    <- corect
directia 2: modific pe BOGDAN (poz 0) sa aiba emailul lui ANDREI (poz 1)
    >>> A TRECUT
studenti cu andrei.ionescu@example.com = 2
a5e-12-32,Bogdan,Popescu,andrei.ionescu@example.com,Pass123,22
c8f-45-71,Andrei,Ionescu,andrei.ionescu@example.com,Secret456,24
```

Două linii în `students.txt` cu același email, scrise pe disc de `save()` .

DE CE cedează într-o direcție și nu în cealaltă. Când iei studentul cu `findById()` , primești **obiectul din listă**, nu o copie. Îi pui emailul nou, deci obiectul **din listă** are deja emailul nou *înainte* ca `update()` să apuce să verifice ceva. Apoi `findByEmail()` pornește de la index 0 și returnează **primul** obiect cu emailul căutat.

- Dacă victima e înaintea lui în listă (direcția 1), primul găsit e victima → id diferit → excepție. Pare că merge.
- Dacă el e primul în listă (direcția 2), `findByEmail` îl găsește **pe el însuși**, deja modificat. `byEmail.get().getId().equals(student.getId())` compară id-ul lui cu id-ul lui → `true` → condiția `!...` e `false` → trece.

Verificarea se compară cu ea însăși. Nu e „aproape corectă”, e corectă doar din noroc de ordonare — și de-asta a trecut de review-ul tău prin citire.

`CourseRepository.update()` (:77-85), scris de tine acum, **n-are deloc** verificare de unicitate — nici măcar cea ruptă. `add()` verifică `existsByCourseName` , `update()` nu. Reprodus:

```
cursuri cu numele 'Java Programming' in fisier:
  C001,Java Programming,Computer Science
  C002,Java Programming,Computer Science
>>> 2 cursuri cu acelasi nume
```

FIX (același în ambele): nu compara id-uri, sari peste obiectul însuși prin identitate de referință. Vezi tabelul Before/After.

Repară-l în amândouă. Apoi caută singur: mai am undeva forma asta?

B6 — RESTANȚĂ: un student șterge cărțile altui student

src/app/View.java:195–216 · src/app/book/repository/BookRepository.java:144

Reprodus. Sunt logat ca Alexandra (m6n-52-67) și cer ștergerea cărții „Refactoring”, care în books.txt aparține lui c8f-45-71 (Andrei):

```
proprietarul lui 'Refactoring' in books.txt = c8f-45-71 (Andrei)
deleteByBookName('Refactoring') = true <- Alexandra e logata
```

deleteByBookName() caută în **toate** cărțile și șterge prima potrivire, indiferent al cui e. View.deleteBook() are student.getId() la îndemână și nu-l folosește.

DE CE contează mai mult decât pare. Ai introdus noțiunea de „studentul curent” în View, dar ea trăiește doar la citire: showStudentBooks() filtrează pe student.getId(), updateBookName() primește studentId. La ștergere, filtrul dispare. Într-o aplicație reală asta nu e un bug de logică, e unul de autorizare: diferența dintre „șterge cartea” și „șterge cartea mea”. Regula pe care o înveți aici e că **fiecare operație care scrie trebuie să răspundă la două întrebări separate** — „există?” și „am voie?”. findByBookName răspunde doar la prima; a doua nu e pusă niciodată.

FIX: deleteByBookNameAndStudent(bookName, studentId), cu condiție dublă. Tiparul îl ai deja scris în findBooksByStudentId() — e aceeași filtrare, mutată la ștergere.

B7 — RESTANȚĂ: orice tastă greșită omoară aplicația

src/app/View.java:43

```
alegere=Integer.parseInt(scanner.nextLine());
```

Reprodus, input `abc` :

```
Exception in thread "main" java.lang.NumberFormatException:
    For input string: "abc"
    at java.base/java.lang.Integer.parseInt(Integer.java:565)
    at app.View.play(View.java:43)
    at app.View.<init>(View.java:31)
    at app.Main.main(Main.java:20)
```

DE CE moare tot programul, nu doar meniul. `Integer.parseInt` aruncă `NumberFormatException`, care e o `RuntimeException` — **unchecked**. Compilatorul nu te obligă s-o prinzi, deci n-ai fost avertizat. Neprinsă, ea urcă din `play()` în `View.<init>` (constructorul, pentru că acolo chemi `play()`), de acolo în `main`, iar când ajunge sus fără să fie prinsă, JVM-ul oprește thread-ul. Ai deja `default:` în `switch` pentru cifre invalide — dar `default` se execută abia **după** `parseInt`, deci pentru text nu apucă niciodată să ruleze.

FIX: citește linia, încearcă `parseInt` într-un `try`, iar pe `catch` afișează un mesaj și `continue` — te întoarce în buclă și reafișezi meniul. Vezi Before/After.

B11 — dez-înscriere de la un curs la care nu ești înscris = stack trace

src/app/View.java:144–145

Reprodus, cer „Operating Systems” (nu sunt înscrisă la el):

```
java.util.NoSuchElementException: No value present
    at java.base/java.util.Optional.get(Optional.java:143)
    at app.View.removeCourse(View.java:145)
    at app.View.play(View.java:64)
```

DE CE. `findByIdAndCourseId()` returnează `Optional` — l-ai scris tu, corect, tocmai ca să exprime „poate să nu existe”. Pe linia următoare chemi `.get()` fără să întrebi `isPresent()`, adică arunci exact informația pentru care există `Optional`. `Optional.get()` pe gol nu returnează `null`, aruncă `NoSuchElementException`.

Nu se închide aplicația (ai `catch (Exception e)` în jur), dar `e.printStackTrace()` îți arată omului un stack trace Java pentru cazul cel mai banal cu puțință: a scris un curs la care nu e înscris. Un `Optional` prins cu `catch` în loc de `if` e un `if` scris cu excepții — și costă de o mie de ori mai mult.

FIX: `if (enrollmentToRemove.isEmpty()) { println("Nu esti inscris..."); return; }.`
Același lucru la `foundCourseByName` (:143) și `foundCourse` (:119) — ambele au `isPresent()`, dar când e `false` nu spun nimic: tastezi un curs inexistent și programul te întoarce mut în meniu.

B12 — `cardRepository.save()` scrie obiecte, nu date

`src/app/cardStudent/repository/cardRepository.java:65`

```
content.append(card.toString()).append(System.lineSeparator());
```

Nu se poate reproduce prin meniu — `save()` e `private` și nicio metodă publică nu-l cheamă încă, iar `cards.txt` e gol. Dar ce ar scrie se vede direct:

```
card.toText()    -> CARD01,1234-5678,a5e-12-32
card.toString()  -> app.cardStudent.model.CardStudent@232204a1
```

DE CE nu te avertizează nimic. `toString()` există pe **orice** obiect Java, moștenit din `Object` — deci `card.toString()` compilează întotdeauna, indiferent dacă l-ai definit sau nu. Tu ai definit `toText()`, nu `toString()`, deci se cheamă cel din `Object`, care returnează `numeClasă@hashCode`. La prima carte de student salvată, `cards.txt` se umple cu adrese de memorie, iar `loadData()` de la următoarea pornire crapă pe `arr[1]` — datele sunt pierdute definitiv, pentru că `save()` rescrie tot fișierul.

Alternativa curată e să faci `toText()` să **dispară** și să pui `@Override public String toString()` peste tot — atunci greșeala asta devine imposibilă. Dar alege una și ține-o în toate cele 5 modele; acum ai `toText()` în 4 și `toString()` folosit în 1.

FIX: `card.toText()`. Și rulează `Ctrl+Shift+F` pe `.toString()` în tot proiectul.

B8 — RESTANȚĂ: o linie în formatul vechi și aplicația nu mai pornește deloc

```
src/app/book/Book.java:21 ( this.setStudentId(arr[3]) )
```

Pe o linie de 3 câmpuri (formatul de dinainte de `a29bce4`):

```
java.lang.ArrayIndexOutOfBoundsException: Index 3 out of bounds for length 3
```

Ai migrat `books.txt` la 4 coloane, dar constructorul cere necondiționat `arr[3]`. `loadData()` prinde `RuntimeException` și îl re-aruncă → `BookRepository` nu se poate construi → `View` nu se poate construi → aplicația nu pornește. O singură linie greșită, scrisă de mână, blochează totul. Același lucru la `Enrollment.java:20-25` (`arr[2]`, `LocalDate.parse`) și `CardStudent.java:12-17` (`arr[2]`).

FIX: în fiecare constructor-din-text, verifică `arr.length` întâi și aruncă un mesaj care spune ce lipsește: `if (arr.length != 4) throw new IllegalArgumentException("Book needs 4 fields, got " + arr.length + ": " + text);`

● Importante

- **M13 — enrollments.txt are deja duplicate, iar loadData() le acceptă.** Reprodus: (`j1k-74-19, C001`) pe liniile 5 și 8, (`t8v-16-92, C002`) pe 3 și 9, (`t8v-16-92, C003`) pe 4 și 10. `addEnrollment()` verifică unicitatea, `loadData()` nu — deci regula ta e apărută doar la intrarea prin cod, nu și la intrarea prin fișier. Efectul se vede la ștergere: `deleteById(t8v-16-92, C002)` scoate o înregistrare și o lasă pe a doua, deci după „dez-înscrisere” utilizatorul e tot înscris. Fie cureți fișierul, fie `loadData()` sare peste perechea deja încărcată.
- **M6 — unicitatea numelui de carte e la nivel greșit.** Reprodus: Alexandra cere „Effective Java” → `Book is already in the store.`, pentru că o are Andrei. Într-o școală, două exemplare din aceeași carte e normalul. Și e **inconsecvent cu tine însuși**: `addBook` verifică global, `updateBookName` verifică doar în cărțile studentului. Aceeași regulă, două înțelesuri, în aceeași clasă. **Decizie de luat — spune-mi care e intenția.** (Ridicat în runda 3 ca M4, netratat.)
- **M9 — studentul curent e hardcodat, iar acum e și mai vizibil.** `View.java:25`: `new Student("m6n-52-67,Alexandra,...")` — o copie a liniei 8 din `students.txt`, ținută sincronizată manual. Pe linia 28 instanțiezi `StudentRepository` și nu-l folosești niciodată. Ai deja `findById()` scris: `studentRepository.findById("m6n-52-67").orElseThrow(...)`.

- **M7 — `e.printStackTrace()` în fața utilizatorului.** `View.java:125, 148, 168, 188, 213` . Omul primește un stack trace pentru că a scris un titlu care există deja. `catch` e locul unde traduci eroarea tehnică în propoziție: `System.out.println(e.getMessage())` .
 - **M8 — `View()` face treabă în constructor.** `View.java:31 : this.play()` . Un constructor construiește obiectul, nu îl și pornește. Efectul se vede în stack trace-ul de la B7: `View.<init>` apare între `main` și `play` . `Main` ar trebui să facă `new View().play()` — acum `Main` nu face nimic vizibil.
 - **M14 — două metode identice în `EnrollmentRepository` .** `findAllEnrollmentsByStudentId` (`:66`) și `findEnrollmentsByStudentId` (`:105`) fac același lucru; diferă doar prin `equalsIgnoreCase` vs `equals` . Plus `indexOfEnrollmentById` (`:118`) care caută după `studentId` , deși numele spune „ById” — nefolosită, dar e o capcană pusă pentru tine peste trei luni. Șterge două din trei.
 - **M11 — meniul nu spune cum se iese.** `View.meniu()` afișează 1–7. `0` e tratat în `switch` , dar utilizatorul n-are de unde să-l ghicească.
 - **M12 — `showStudentBooks()` tace când nu are ce arăta.** Listă goală → zero output → omul nu știe dacă a mers și n-are cărți, sau n-a mers. Idem `showCoursesByEnrollments` .
 - **M10 — cod orfan după refactorizări.** `Course.registeredStudents` (`Course.java:14`) e declarat și niciodată populat — rest din relația bidirecțională pe care ai scos-o corect la B5. `View.showCoursesByEnrollmentIds` (`:237`) e o metodă goală. `cardRepository` n-are nicio metodă de scriere și nu e instanțiat nicăieri.
-

Cleanups

- **C8 — typo propagat: `stundentId` .** `Book.java` (7 apariții), `BookRepository.java` (2). `setStundentId` / `getStundentId` sunt acum API public — cu cât întârzii redenumirea, cu atât mai multe locuri o folosesc. IntelliJ: `Shift+F6` pe câmp.
- **C12 — clasă cu literă mică: `cardRepository` .** Singura din proiect. În Java, clasele încep cu majusculă — `CardRepository` . Fișierul se redenumește odată cu ea.
- **C9 — importuri parazite din autocomplete,** nefolosite și absurde în context: `import javax.swing.text.html.Option;` (`BookRepository.java:6`) și `import javax.smartcardio.Card;` (`cardRepository.java:6`). Apar când accepți prima sugestie la `Option` / `Card` . `Ctrl+Alt+O` = optimize imports. La fel: `Enrollment.java:3` importă propriul repository, `Main.java:4–8` importă 5 clase și nu folosește niciuna.

- **C10** — `import app.student.model.Student` rămas în `Book.java:3` și în `Course.java:3`, `CardStudent.java:3` — nefolosite după ce ai scos relațiile.
 - **C11** — **două API-uri de ștergere inconsistente:** `deleteByBookId` aruncă excepție, `deleteByBookName` returnează `boolean`. Aceeași clasă, două convenții.
 - **C3, C4, M5 din runda 3** — **neatinse:** `IllegalArgumentException` la erori de I/O (ar fi `IllegalStateException` — `CourseRepository` și `StudentRepository` o fac deja corect, `BookRepository` și `EnrollmentRepository` nu), `get/setCreated_at` snake_case, `Book` în `app.book` în loc de `app.book.model` (singurul model care nu e în `model/`).
-

Before / After

B10 — modificarea trebuie să ajungă pe disc

Acum (`BookRepository.java:107,124`):

```
List<Book> books = findBooksByStudentId(studentId);  
...  
book.setBookName(newName);  
return book;
```

Cum ar trebui:

```
List<Book> studentBooks = findBooksByStudentId(studentId);  
...  
book.setBookName(newName);  
save();  
return book;
```

Regula: **orice metodă care schimbă starea se termină cu `save()`**. Uită-te acum la toate metodele tale de scriere din toate cele 4 repository-uri și verifică una câte una.

B2 — validare care nu se poate ocoli

Acum (`StudentRepository.java:69-74`):

```
Optional<Student> byEmail = findByEmail(student.getEmail());
if (byEmail.isPresent()
    && !byEmail.get().getId().equals(student.getId())) {
    throw new IllegalArgumentException(
        "Email already used: " + student.getEmail());
}
```

Cum ar trebui:

```
for (Student other : students) {
    if (other == student) continue;
    if (other.getEmail().equalsIgnoreCase(student.getEmail())) {
        throw new IllegalArgumentException(
            "Email already used: " + student.getEmail());
    }
}
```

`other == student` sare peste obiectul însuși prin **identitate de referință** — nu prin id. De-asta merge indiferent de ordinea din listă: nu mai există niciun caz în care obiectul se compară cu el însuși. Aceeași buclă, cu `getCourseName()`, se adaugă în `CourseRepository.update()`, unde acum nu există nimic.

B9 — butonul face ce scrie pe el

Acum (`View.java:63–65, 88`):

```
System.out.println("7-inscrie-te la un curs:");
...
case 7:
    removeCourse();
    break;
```

Cum ar trebui:

```
System.out.println("7-inscrie-te la un curs");
System.out.println("8-renunta la un curs");
...
case 7:
    registerForCourse();
    break;
case 8:
    removeCourse();
    break;
```

B6 — ștergi doar ce e al tău

Acum ([View.java:205](#)):

```
boolean erased = bookRepository.deleteByBookName(bookName);
```

Cum ar trebui:

```
boolean erased = bookRepository.deleteByBookNameAndStudent(
    bookName, student.getId());
```

Iar în repository, condiția devine dublă:

```
if (book.getBookName().equalsIgnoreCase(bookName)
    && book.getStudentId().equals(studentId)) { ... }
```

B7 — meniul nu are voie să moară

Acum ([View.java:43](#)):

```
alegere = Integer.parseInt(scanner.nextLine());
```

Cum ar trebui:

```
String linie = scanner.nextLine().trim();
try {
    alegere = Integer.parseInt(linie);
} catch (NumberFormatException ex) {
    System.out.println("Enter a number.");
    continue;
}
```

`continue` te întoarce în buclă și reafișezi meniul — exact rolul lui `default` din `switch`, dar pentru input care nici măcar nu e număr.

B11 — **Optional** se întreabă, nu se prinde cu `catch`

Acum ([View.java:144–145](#)):

```
Optional<Enrollment> toRemove = enrollmentRepository
    .findByIdAndCourseId(student.getId(), courseId);
enrollmentRepository.deleteById(
    toRemove.get().getStudentId(),
    toRemove.get().getCourseId());
```

Cum ar trebui:

```
Optional<Enrollment> toRemove = enrollmentRepository
    .findByIdAndCourseId(student.getId(), courseId);
if (toRemove.isEmpty()) {
    System.out.println("Nu esti inscris la acest curs.");
    return;
}
enrollmentRepository.deleteById(
    toRemove.get().getStudentId(),
    toRemove.get().getCourseId());
System.out.println("Dezinscris.");
```

Q&A — verificare de înțelegere

1. **B10.** `findBooksByStudentId()` returnează o listă nouă, dar `book.setBookName()` chiar modifică obiectul din repository. Explică de ce amândouă sunt adevărate în același timp. Ce anume se copiază când faci `new ArrayList<>(books)` — și ce nu?
 2. **B2.** Verificarea din `update()` cedează într-o direcție și ține în cealaltă, în funcție de cine e primul în listă. Spune, în propriile cuvinte, **în ce moment** obiectul din listă capătă emailul nou — înainte sau după ce `update()` verifică? De ce răspunsul la întrebarea asta explică tot bugul?
 3. **B9 + B6 + M9.** `View` știe cine e studentul curent și, la scriere, nu-l folosește. Fă lista completă: care operații din `View` ar trebui să depindă de `student.getId()` și nu depind? (Sunt mai multe decât cea din B6.) Și de ce `registerForCourse()`, care îl folosește corect, nu e chemată niciodată?
-

Ordinea de lucru

1. **B9** — meniul face invers decât promite. Un utilizator care încearcă să se înscrie se dez-inscrie. Zece minute de reparat, e cel mai vizibil defect din aplicație.
 2. **B10** — `save()` lipsă. Pierdere de date, și e o regresie față de codul tău vechi.
 3. **B2** — în amândouă repository-urile. A doua rundă, și e singurul care corupe date deja scrise pe disc.
 4. **B7 + B11** — programul nu are voie să moară și nici să arate stack trace-uri.
 5. **B6** — filtrarea pe student la ștergere; apoi Q3, restul operațiilor.
 6. **B12 + B8 + M13** — datele: `toText()`, `arr.length`, duplicatele din fișier.
 7. 🟡/🟢 la final. C8 și C12 (redenumirile) merită făcute primele, cât sunt ieftine.
-

Regula rundeii: `EnrollmentRepository` arată că știi să scrii verificarea corectă — ai pus-o singur, fără s-o cer, în `addEnrollment()`. Aceeași verificare lipsește din `CourseRepository.update()`, scris de tine în aceeași săptămână. Deci nu e o problemă de cunoștințe, e una de **recitare**: când termini o clasă, deschide sora ei și compară-le metodă cu metodă. B10 e același lucru — ai avut `save()` scris, l-ai șters într-un refactor și n-ai recitat. **Codul care „merge la rulare” și codul corect nu sunt același lucru; diferența o vezi doar dacă o cauți.**

Runda 3 — commit `a92b793` (istoric)

Data: 2026-08-19 · Relații între entități peste stratul de repository.

-  **B1** `Main` crapă la orice rulare — închis
 -  **B2** `update()` acceptă duplicate și le salvează — **NEATINS** (vezi runda 4)
 -  **B3** `CourseRepository` nu încarcă niciodată datele — închis
 -  **B4** `addBook()` aruncă id-ul primit — închis
 -  **B5** aceeași carte la doi studenți — închis prin trecerea la cheie străină
 -  **M1** relațiile nu se persistau — închis (a 4-a coloană în `books.txt`)
 -  **M4** unicitate globală pe numele cărții — netratat (revine ca M6)
 -  **M5** `Book` în `app.book` în loc de `app.book.model` — neatins
 -  **C3** `IllegalArgumentException` la erori de I/O — parțial (2 din 4 repo-uri)
 -  **C4** `get/setCreated_at` snake_case — neatins
 -  **C1** câmpuri de test în `Student` — închis
-

Runda 2 și 1 (istoric)

Stratul de repository: `findById / findByX` cu `Optional`, `existsByX`, `indexOf` cu `-1`, `save()` care rescrie tot fișierul, `loadData()` cu număr de linie în eroare. Tiparul a fost internalizat și aplicat de patru ori, singur.



MyCodeSchool

Material didactic MyCodeSchool

mycodeschool.ro